

ADA

Algoritmer & Datastrukturer

Introduktion mm.

7. september, 2026

CV – Ole Dolriis

1953	Født i København
1976-1977	Programmør Danfoss
1977-1979	Programmør Regnecentralen
1979-1983	Softwareudvikler og team lead ELSAM (Ørsted)
1983-1989	Softwareudvikler og team lead Mærsk Data
1989-1993	Projektchef Mærsk Data
1993-2001	Underviser TietgenSkolen (Datamatikeruddannelsen)
2001-2006	Lektor Ingeniørhøjskolen Odense Teknikum
2006-	Lektor/Docent SDU (studieleder 2005-2020)
2024-2026	Adjunktvikar Erhvervsakademi København (Datamatiker)

Education

High School Graduate	1972	Pennsylvania, USA
Student (mat./fys.)	1974	Mulernes Legatskole
Edb-Assistent	1976	TietgenSkolen
HD-R(regnskabsvæsen)	1986	Odense Universitet
Lederuddannelse	1991	Mærsk Data
Bachelor i datalogi	1998	Aalborg Universitet
Cand.scient. i datalogi	2007	Aalborg Universitet
Cand.mag. i historie	2016	Syddansk Universitet

Dengang

Mandag den 15. marts 1976 tiltrådte jeg min første stilling som programmør på Danfoss i Nordborg.

Sammen med 25 kollegaer skulle jeg udvikle Edb-systemer på en IBM-mainframe System370. Den havde 2 megabyte RAM, fyldte en gymnastiksal og kostede \$70.000 om måneden at lease – den kunne ikke købes. Min månedsløn var 4.600 kr.

Programmer blev skrevet på kodeark og sendt til hulning i hullestuen. Tidligst dagen efter fik man en programudskrift (postliste) retur med eventuelle kompileringsfejl. Efterfølgende rettelser blev skrevet på hulkort, som blev indlæst sammen med jobkort på en lokal kortlæser (mainframen stod i en anden bygning). På en god dag kunne man afvikle tre testskud. Skærme fandtes kun i maskinstuen.

Det har i sandhed (og for en gangs skyld) været en lang rejse. Jeg tvivler på, at nogen anden branche kan fremvise lignende fremskridt og ændringer.

IBM 370



alamy

Image ID: FF7GEA
www.alamy.com

Instruktor

Mads Willum Bech

mach018@student.sdu.dk



Uddannelser

Softwareteknologi (diplom)

Robotteknologi (diplom)

Robotteknologi (civil)

Spiludvikling og Læringsteknologi (diplom)

Elektronik (diplom)

Fra kursusbeskrivelsen

Læringsmål:

Viden

Efter at have gennemført kurset har den succesfulde studerende viden om:

- Datastrukturer (f.eks. Liste, Stak, Kø, Træ, Binært Søgetræ, AVL-træ, Heap, Graf)
- Algoritmeprincipper (f.eks. Greedy, Divide and conquer), dynamisk programmering
- Komplexitetsanalyse af datastrukturer og algoritmer, herunder rekurrens
- Problemer og eksempler i relation til datastrukturer og algoritmer
- Implementering og sproglige konstruktioner i objektorienteret programmeringssprog

Færdigheder

Efter at have gennemført kurset har den succesfulde studerende:

- Styrket programmeringserfaring i et objektorienteret sprog

Kompetencer

Efter at have gennemført kurset kan den succesfulde studerende:

- Implementere de gennemgåede algoritmer og datastrukturer på optimal vis i forhold til tids- og pladsforbrug.

Praktisk

Lektioner:

48

Undervisningsform:

Forelæsninger samt opgaveløsning med vejledning.

Eksamensbetingelser:

Godkendt opgaveportefølje.

Eksamen:

3-timers skriftlig eksamen med offline hjælpemidler; ingen censur; karakter efter 7-trinsskalaen.

Forudsætninger

Grundlæggende objektorienterede programmeringskompetencer i Java, C# eller C++.

Spørgsmål

- Hvor rusten er du i OO-programmering?
- Hvornår brugte du sidst tid på at programmere?

Råd

Et vigtigt læringsmål på dette kursus er at forbedre dine generelle programmeringskompetencer, så lad være med at søge hjælp på nettet, med mindre alle andre muligheder er udtømte.

Og løs samtlige programmeringsøvelser. De fleste, som dumper kurset, har utilstrækkelige programmeringsfærdigheder.

Lærebøger

Mark Allen Weiss

Data Structures and Algorithm Analysis in C++
(4th ed., 2014)

Data Structures and Algorithm Analysis in Java
(3rd ed., 2012)

Weiss' bemærkninger om computer performance er forældede af indlysende årsager.

Om lærebøgerne

Lærebøgerne er designet til at kunne bruges på et bachelor **og** et kandidatkursus, og derfor er dele af teksten ikke relevant for dette kursus.

Der vil i nogen grad blive tale om *cherrypicking*.

Vi tager lettere afslappet på matematikken, især i de senere kapitler.

Mere om lærebøgerne

Lærebøgerne er ikke helt nye; men det er lidt lige som med matematikbøger. Fagets pensum har ikke ændret sig ret meget i de seneste årtier, så en ny lærebog hver andet år giver ikke mening.

Giv lærebogen en chance

- Jeg er sikker på, at tidligere studerende har bestået kurset uden at have læst så meget som én linje i lærebogen.
- Hvis man finder Pixie-ting om emnerne på nettet og løser opgaverne, kan man muligvis bestå eksamen.
- Men når man har vænnet sig til Weiss' stil, tilbyder han en lang række værdifulde indsigter.
- Og når han kammer over med matematikken, så gå trygt videre til næste afsnit (med mindre jeg har sagt noget andet).

Foreløbig pensumliste

Kapitel 1	1.1-1.3
Kapitel 2	2.1 og især 2.4 (minus 2.4.3)
Kapitel 3	baggrundsstof
Kapitel 4	4.1-4.4 og 4.6
Kapitel 5	5.1-5.5 og 5.7 (hop)
Kapitel 6	6.1-6.4
Kapitel 7	7.1-7.7 undtagen matematikken i 7.4
Kapitel 9	9.1-9.3.2; 9.5; 9.7 ekstensivt
Kapitel 10	10.1

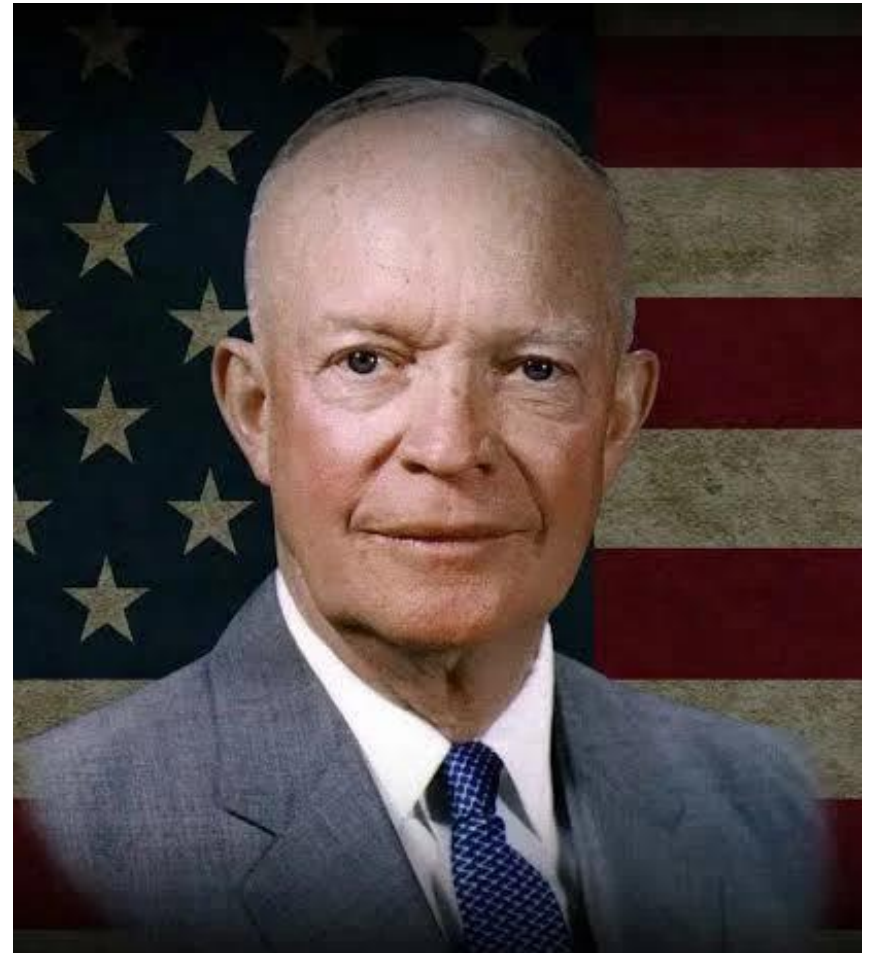
Foreløbig plan (6. sept.) – Lokale U150 – 12:15-15:45

1. Introduktion; matematikken; rekursion	Kapitel 1 og 2	7. sept.
2. Store-O; kø, stak og liste	Kapitel 2 og 3	14. sept
3. Hashing	Kapitel 5	21. sept.
4. Prioritetskøer	Kapitel 6	28. sept.
5. Sortering	Kapitel 7	5. okt.
6. Sortering forts.	Kapitel 7	19. okt.
7. Træer	Kapitel 4	26. okt.
8. Træer forts.	Kapitel 4	2. nov.
9. Disjunkte mængder og grafer	Kapitel 8 og 9	9. nov.
10. Grafer forts.	Kapitel 9	16. nov.
11. Designteknikker	Kapitel 10	23. nov.
12. Repetition og eksamensforberedelse		30. nov.

Citat

”Battle planning is important, but as soon as the first shot is fired, your plan goes up in smoke.”

Dwight D. Eisenhower (”Ike”, 1890-1969) – Amerikansk general under 2. verdenskrig og den 34. præsident (1953-1961).



Udviklingsværktøjer (IDE)

Jeg bruger:

C++/C#

Visual Studio

Java

NetBeans/IntelliJ



Om programmering

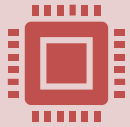
- Omtanke/refleksion
- Præcision
- Tålmodighed
- Disciplin

Hvad er en algoritme?

En algoritme er en endelig sekvens af veldefinerede af computer-implementerbare instruktioner, typisk til løsning af en klasse af problemer eller til at udføre en beregning.

Wikipedia, som citerer diverse videnskabelige kilder.

Semantisk overvejelse



I de senere år er begrebet *algoritme* blevet anvendt i et stigende antal meget forskellige sammenhænge.



Og dermed er betydningen af ordet blevet udvandet.



Nogle personer argumenterer for, at AI beskæftiger sig med problemer, som **ikke** kan løses af algoritmer.

Et typisk problem, som kan løses af en algoritme

- At skrive et program, som kan finde det k -største tal i en tabel af heltal er ikke særlig svært.
- Man kan fx sortere tabellen og returnere det k . element.
- Det er bare ikke særlig optimalt, hvis tabellen er meget stor.
- Vi bliver nødt til at finde på en smartere løsning, hvis tiden er en kritisk faktor.

Hvad er en datastruktur?

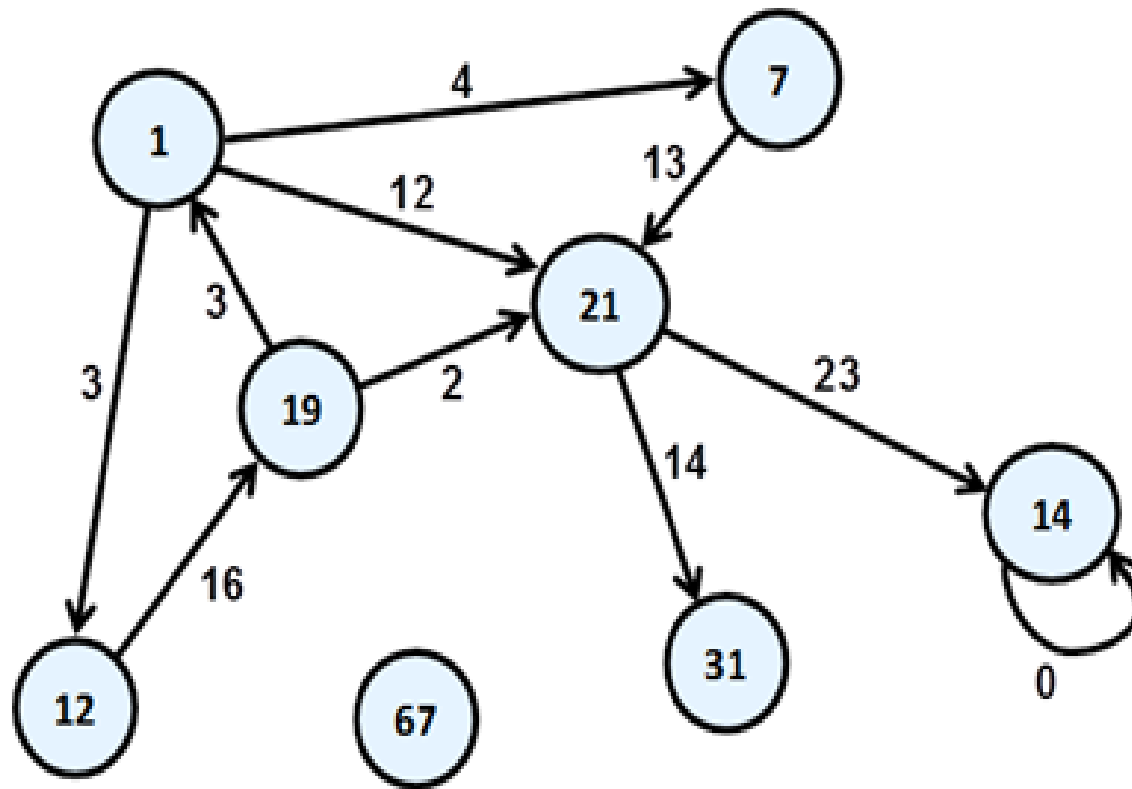
En datastruktur er organisering, håndtering og lagring af data, som tilsikrer effektiv tilgang og modificering.

Mere præcist er en datastruktur en samling af værdier, relationerne imellem dem og de funktioner, som kan udføres på data.

Wikipedia, som citerer diverse videnskabelige kilder.

PS Bemærk ligheden med definitionen af en *klasse* i OOP.

Data struktur - graph



Et graf-problem

Broerne i Königsberg



Findes der en sti gennem byen, hvor hver bro
passeres præcis én gang?

Hvad så med ChatGPT og andre LLM's?

- De vil kunne hjælpe dig med de fleste opgaver.
- Nogle af dem vil de kunne løse til perfektion.
- Af og til vil de give forkerte svar.
- Mit råd er: anvend dem med forsigtighed.
- De vil **ikke** være tilgængelige til eksamen, heller ikke lokale.
- Hverken fakultetet eller universitetet har udmeldt specielt brugbare retningslinjer for anvendelse af LLM's.

Matematikken

Matematikken i dette fag er bl.a. karakteriseret ved:

- Halvering af problemer -> rekurrens
- Anvendelse af logaritmer
- Altid totals-logaritmer
- En del summationer 😊
- Induktionsbeviser
 - Summen af de første N naturlige tal er $N*(N+1)/2$.

Induktionsbeviser

- En algoritme opererer sædvanlig vis på et antal elementer.
- Vi kalder det tal for N , og N er selvklart et *naturligt tal*.
- I induktionsbeviset ***antager*** vi, at udsagnet virker for N elementer.
- Hvis vi kan vise, at det også virker for $N+1$ elementer, er vi på den sikre side, eftersom N kan antage alle positive værdier.

Hvorfor totals-logaritmer?

Fordi problemer ofte halveres på vej mod løsningen.

Eksempel: binær søgning – at finde eller ikke finde et element i en tabel af sorterede heltal (skrevet i Java):

```
public static boolean binarySearch(int[] array, int value)
{
    int begin = 0; int mid = 0; int end = array.length-1;
    while (begin <= end)
    {
        mid = (begin + end) / 2;
        if (array[mid] < value)
            begin = mid + 1;
        else
            if (array[mid] > value)
                end = mid - 1;
            else
                return true;
    }
    return false;
}
```

Rekursion – en metode der kalder sig selv

```
public static int f(int x)
{
    if (x == 0)
        return 0;
    else
        return 2*f(x-1) + x*x;
}
```

- Basis
- Gør fremskridt mod basis
- Problemet reduceres på vej ned (del/divide)
- Løsningen skabes på vej tilbage (hersk/conquer)

Forløb

x = 5 : 2 * f (4) + 25 -> 141

x = 4 : 2 * f (3) + 16 -> 58

x = 3 : 2 * f (2) + 9 -> 21

x = 2 : 2 * f (1) + 4 -> 6

x = 1 : 2 * f (0) + 1 -> 1

x = 0 : -> 0

Mere rekursion

```
public static int sumNaturalNumbers(int n)
{
    if (n < 1)
        return 0;
    return n + sumNaturalNumbers(n-1);
}
```

1. Base case
2. Arbejd mod base case.
3. Designregel: antag at det rekursive trin virker.
4. Undgå at løse den samme instans af problemet flere gange.

Fordele ved anvendelse af rekursion

Man behøver ikke at holde styr på alle de rekursive skridt, hvilket kan være en kompliceret affære.

Dette håndteres af en mekanisme i operativsystemet kaldet runtime-stakken, som holder styr på de 'nestede' metodekald ved hjælp af såkaldte *activation records*.

Nogle hævder desuden, at rekursive metoder er nemmere at forstå end iterative. Andre er uenige.

Endnu et eksempel på rekursion

```
public void walk( String path )
{
    File root = new File( path );
    File[] list = root.listFiles();

    if (list == null) return;

    for ( File f : list ) {
        if ( f.isDirectory() )
        {
            walk( f.getAbsolutePath() );
            maps++;
        }
        else
            files++;
    }
}
```

Verifikation

Rekursive algoritmer er sjældne eksempler på, at det kan bevises matematisk, at et program er fejlfrit. Det skyldes deres induktive natur.

Med større programmer er fuldstændig verifikation umulig helt i overensstemmelse med påstanden, om at **et program er kun fejlfrit, indtil man finder den næste fejl.**

N

angiver antallet af elementer

- som skal sorteres
- som skal indsættes i en liste
- noder/kanter i en graf
- i en lists/array som skal gennemløbes

Tommelfingerregler

- Halvering af problem $O(\log N)$
- En for-løkke: $O(N)$
- To for-løkker i sekvens: $O(N)$
- for-løkke med indb. halvering: $O(N \log N)$
- To nestede for-løkker: $O(N^2)$
- Tre nestede for-løkker: $O(N^3)$
- Tjek alle kombinationer: $O(2^N)$

Forskellige tilfælde

Best

Average

Worst

Det næste skridt

Algoritmer

vs.

Kunstig intelligens (AI)

Hvad kendetegner AI?



Masser af definitioner.



Min favorit foreslår, at AI tager sig af problemer, der er så svære eller komplekse, at de ikke kan løses af algoritmer.



Definitionen tager afsæt i, at algoritmer er *deterministiske* (verificerbart resultat), mens AI er *probabilistisk* (resultat med en sandsynlighed).



Men den er formentlig forældet.



Nu om stunder tales der ofte om algoritmer, når der diskuteres AI.

Et old school eksempel: ekspertsystemer

Min kone og den ukendte fugl på stranden,
som viste sig at være en strandskade.

”Den var på størrelse med en måge og lignede en måge.”



Den mindste måge i danske farvande er *hættemågen*, som måler 40 cm.



Den største måge i danske farvande er *svartbagen*, som måler 70 cm.

”Dens hoved var sort”



Hættemåge
Larus ridibundus

”Dens ryg var sort”



Sildemåge
Larus fuscus

”Dens næb var rødt”

Strandskade

Haematopus ostralegus



En AI metode eller to



En af AI's mest prominente (GOFAI) metoder opererer efter devisen *"hvis den ligner en and og går som en and, så er det sgu' nok en and."*

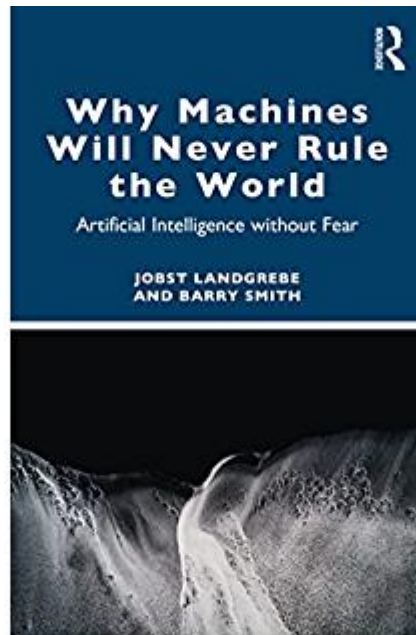


I dette tilfælde lignede den en måge, men var ikke en måge, så her duede metoden ikke.



Nu om stunder ville en oplagt løsningsmodel være billedgenkendelse; men det er vigtigt at huske på, at der i ingen af tilfældene er tale om *sikre* svar, men kun løsningsforslag med en større eller mindre *sandsynlighed*.

Relevant litteratur (2023)



Mit kondensat

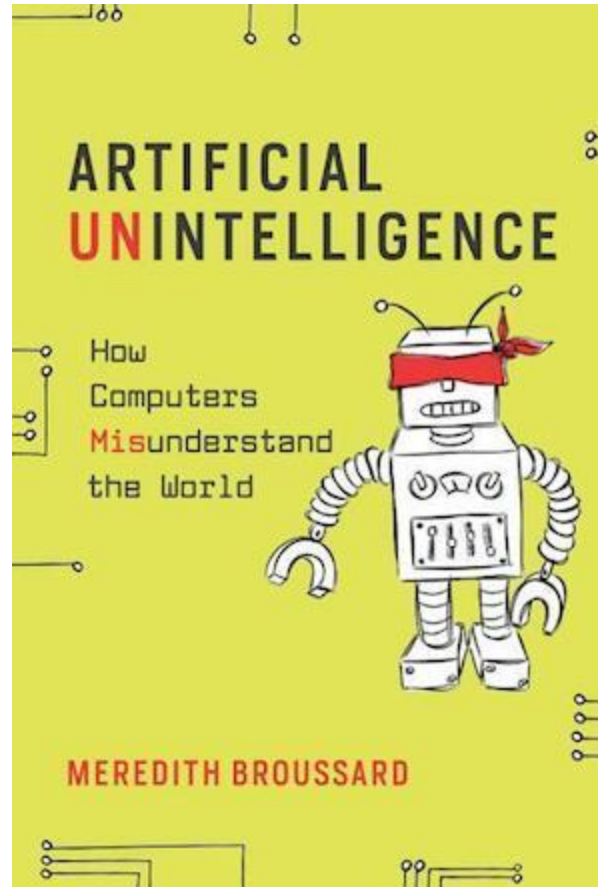
Landgrebe og Smiths måske vigtigste pointe er, at hvis man skal fremstille en fysisk genstand, skal den først modelleres matematisk.

På grund af sin plasticitet kan den menneskelige hjerne ikke modelleres matematisk.

Konklusion: glem AGI og Turing-testen.

Djævelens advokat: hvad så med de forskellige typer af læring, som i høj grad kendetegner moderne AI?

Forældet litteratur (2018)



Dagens (forældede) citat

”We have a small, elite group of men who tend to overestimate their mathematical abilities, who have systematically excluded women and people of color in favor of machines for centuries, who tend to want to make science fiction real, who have little regard of social convention, who don’t believe that social norms or rules apply to them, who have unused piles of money sitting around, and who have adopted the ideological rhetoric of far-right libertarian anarcho-capitalists.”

Meredith Broussard
Artificial Unintelligence
How Computers Misunderstand the World
MIT Press, 2018, p. 85

Mislykket forsøg

Sammen med Prof. Norbert Krüger begyndte jeg for nogle år siden at skrive en bog (på tysk) om kunstig intelligens' historie. Vi opgav efter et par år, fordi vi indså, at bogen ville være håbløst forældet, inden vi var færdige med at skrive den.



Øvelser - primære

- Skriv en rekursiv algoritme med følgende signatur: `int logTwo(int N)` Algoritmen returnerer totals-logaritmen af N, og det er en precondition, at N er et naturligt tal og en potens af 2. Kaldt med `N = 32` returneres 5, og med `N = 4096` returneres 12.
- Skriv en rekursiv metode, som har en string og en char som parameter og returnerer det antal gange char forekommer i string. Kaldt med `"banana"` returneres 3.

Øvelser - primære

Løs følgende opgaver med rekursion:

```
int sum(int n);    //returns the sum of the first
                  //n odd natural numbers
```

```
int evenSquares(int n); //returns the sum of the first n
                        //even numbers' squares
```

```
int fib(int n);    //returns the nth Fibonacci number
```

```
bool linear(string s, char c, int l);
//returns true if string s with the length l contains
//char c, otherwise false.
```

```
bool binarySearch(int arr[], int value)
//returns true if value is in arr, otherwise false
//the elements in arr are sorted
```

Øvelser - sekundære

Induktionsbeviser:

1. Summen af de første n ulige naturlige tal er n^2
2. Summen af de første n kubiktal er $n^2(\frac{(n+1)^2}{4})$
3. $\sum_1^n \frac{1}{(2n-1)(2n+1)} = \frac{n}{2n+1}$
4. $1 + 3 + 3^2 + \dots + 3^n = \frac{3^{n+1}-1}{2}$ for $n \geq 1$
5. $n = x * 4 + y * 5$ ($n \geq 12; x, y \geq 0$)
6. $7^n - 1$ er dividerbar med 6 for $n > 0$
7. $8^n - 3^n$ er dividerbar med 5 for $n \geq 1$.